

Dealing with a Low-Quality Engineering Culture at Big Tech

Big Tech expectations vs reality, handling the initial shock, and improving the engineering culture.



GERGELY OROSZ

APR 12, 2022 • PAID



78



7



Share



“Q: I’ve joined a FAANG company (one of Facebook, Apple, Amazon, Netflix and Google.). I was hoping to learn more best practices and to improve my engineering skills.

“But a few months in, I’m shocked by the engineering culture. The team I joined has the worst codebase I’ve seen. No unit tests, poor naming, code reviews non-existent, and the build takes 30 minutes. My manager is uninterested in my suggestions to improve things. What can I do? Help!” – a disillusioned software engineer

Dear disillusioned engineer, I’m glad you have shared your shock and concern. While it’s normal that we don’t assume the highest engineering quality at a startup, most of us expect a strong engineering culture to greet us when joining Big Tech for the first time.

While many teams within big companies do have a high engineering bar, this is definitely not universally true. In this issue, we dive deep into this topic:

1. Big Tech expectations vs reality.
2. Reasons for lower engineering standards.
3. Handling the initial shock.

4. Improving the engineering culture.
5. Advice to avoid surprises when joining a new company.

1. Big Tech expectations vs reality

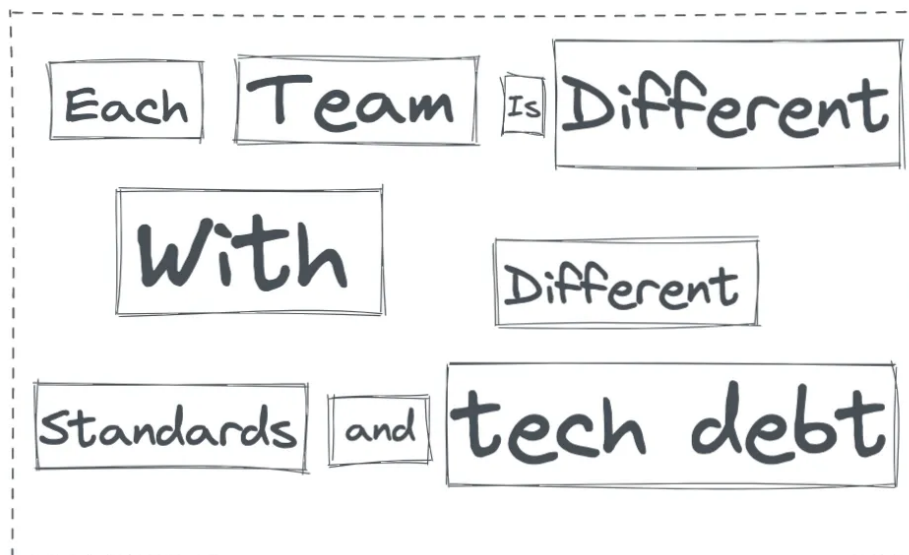
You join a well-known tech company. You've heard only great things about the engineering culture, and after spending lots of time reading through their engineering blog, you are certain this is a place where the engineering bar - its standards - is high, and everyone seems to work on high-impact projects. Yet when you join, reality totally fails to live up to your expectations.

Large companies are not one "unit," as most people imagine. This is likely to be the biggest reason why what you find does not match up to your expectations.

Big Tech Expectation

- ✓ Great Engineering Practices
- ✓ Consistently High Quality
- ✓ Little to no tech debt

vs Reality



pragmaticengineer.com

Expectations vs reality when joining a Big Tech.

In reality, all large companies are made up of several organizations of hundreds of engineers. Each of those organizations is composed of dozens or hundreds of teams,

all of which own their own platforms and products. Every one of these platforms and products is at a different stage of its lifecycle.

There are products which are just being built and are expected to launch in a year's time. Or there are ones where usage is going down, which will be retired in 6-9 months. Then there are highly successful products that were built 10 years ago, which make so much money that they should only be changed with great care. This means some bugs are not fixed, as they are by now accepted as just part of how the product works.

Based on this, visualizing a company as a collection of "bubbles" or "startups" is much closer to reality and more precise. This visualization is more accurate than most mental models.

Take Uber as an example, a company I know very well. There are "bubbles" which care greatly about engineering excellence and scalability, such as the infrastructure platform teams. There are also mature organizations which move slowly as they have proven business models like the Driver or Rider organizations.

There are also newly founded "startups" that move fast and break things – and may be shut down soon if they don't get traction. Uber Air is an example of a now defunct former "startup." Finally, there are teams which came in-house via acquisitions and brought all their legacy code with them, which now need to integrate into the greater organization. JUMP is such an example.

Each team has a different dynamic, even within a single organization. Take the teams below. They're so distinct, yet could all be part of the same organization within a large tech company.

- **The stable senior team.** A team of 12 engineers of whom 8 have worked on the team for more than 3 years. No one has left the team during the past two years. Almost everyone has been promoted to the senior engineer level and above. The team consistently delivers high-impact work and trusts their manager. They manage to find a good balance between getting things done and investing enough in engineering excellence.

- **The newly formed team.** Formed two months ago, this team is expected to build a new Ads product. The team has 10 headcount confirmed, and two engineers transferred internally; they are seeding the team. The manager and the 8 other engineers are currently being hired. Meanwhile, the most senior internal transfer is acting as tech lead. Nothing is set in stone; not on the roadmap, nor in the engineering approach.
- **The junior team.** A team of 6 engineers, all with less than 3 years of experience. The manager is new to the company and has had trouble hiring senior engineers; all candidates have declined to work on this team. The manager is not hands-on, but is someone who believes in empowering engineers. They want to build a team which every engineer loves working on, but the backlog is already full with tasks to refactor parts of the code.
- **The overworked team.** This team used to have 8 engineers, but now is down to 4. Two senior engineers left for better compensation and two others transferred internally because of the overly high oncall load. The team has lots of outages, the manager is unable to attract more candidates, and they are behind on shipping a feature three other teams depend on.
- **The toxic team.** The most senior engineer on the team is avoided by fellow team members as they're overly negative and patronising. Team members have complained to the manager, who is reluctant to upset this engineer, who's seen as highly productive. Oh, and this most senior engineer also happens to think unit testing is a waste of time because 'good engineers don't write buggy code.'
- **"The manager's about to be fired" team.** Unbeknown to team members, the manager of this team is on a Performance Improvement Plan after their own manager was unhappy with their performance and attitude. The manager is worried and their target is to ship a feature on time. Panicking, they instruct engineers to not do any engineering best practices that may cause any slowdown, no matter what the long-term price. The team is puzzled by this. Two engineers silently revolt and decide to ignore these requests, and deliberately take their own sweet time in building out an automation suite.

All these teams and situations occur in large tech companies. When joining a big company, you could find yourself in any one of them. Is it any wonder that team

dynamics and degree of focus on engineering practices will differ wildly?

2. Reasons for a lower engineering bar

When starting at a new company where you observe a low engineering bar, it is natural to assume that something is broken and it should be fixed.

But what if it's something that doesn't need fixing immediately? Or it's a situation that can't be fixed in the short-term? Below are several situations which are often the causes of a subpar engineering culture.

Things that can impact engineering excellence

Product maturity	Team dynamics	Organizational dynamics
Prototyping	Hands-off EM	Acquisition
Pre-PMF	Product / sales running the show	Misaligned incentives
Fighting for survival	No sr engineers	Quality vs promotions
Agressive growth	Stretched team	Inconsistent quality bar
Cash cow	Changing requirements	
To be sunset	External deadlines	

pragmaticengineer.com

Possible causes of subpar engineering culture.

Product maturity and lifecycle

The maturity of the product a team owns matters a lot when looking at how important engineering practices are on this team. Here are a few levels of product maturity.

Prototyping phase. If a product is in such an early phase that only prototyping is underway, there's little point investing in quality practices. This assumes the code

written is to be throwaway and used only for experimental purposes.

A personal example of this is when I joined the Skype for Xbox One team in 2012. It was more than a year before the Xbox One launch and we spent the first few months doing throwaway prototypes on a very early version of the Xbox One. We were focused on getting things done, and didn't care about code quality – yet.

Product before product-market fit (PMF). If a product has not yet made a business case to prove that it will keep attracting funding, teams are better off iterating quickly instead of investing in practices for a product which might get shut down.

For example, this is how many new organizations started within Uber, like UberEats, UberAir or UberRush. If they failed to get enough traction, they were shut down. In their early phases, all these teams wrote quick and dirty code and focused on shipping rapidly.

This approach made sense in all cases. For UberRush, why would the team invest more effort in excellence, when the product was shut down because there was no great business fit? And in the case of UberEats, once the team found the product-market fit, they were able to make the case to invest more in engineering practices. It made sense, once the future of the product was clear.

A product fighting for survival. When products start to miss the business metrics they should hit, they become at risk of their headcount getting reduced, or being shut down altogether. These business metrics might be the number of active users, revenue generated or any similar ones.

When the team's existence is at risk, longer term investments like engineering excellence tends to take a back seat, while shipping features that could turn things around becomes a priority.

I've observed this many times. Most teams in such a situation have either not yet found the product-market-fit, or are teams which were born via an acquisition and have aggressive metrics to hit, which they are not doing.

Interestingly enough, colleagues on neighboring teams often miss the context that a team like this is fighting for survival. Instead, the observer's focus is on the lower code quality, or the rushed engineering decisions the team makes.

Aggressive growth. On teams that have seen a large number of engineers join, new recruits might all have a different sense of what "quality" looks like. It's normal to observe lower quality after an aggressive growth phase. However, this should change as the team beds in.

A "cash cow" product which is risky to change. If a team owns a product which is generating massive profit, but the code is fragile and risky to change, there are teams whose job is to keep it this way. However, they lack technical leadership and are doing themselves a disservice. If anything, a cash-cow product is its own argument for investment in resilience, for upgrading the architecture and practices so the money-spinning product is cheaper to maintain and costly outages are less frequent.

However, coupled with a product-led leadership, and outages in the past which derived from refactoring or removal of technical debt, some teams adopt the low-quality stance of "if it ain't broke, don't fix it".

A product to be sunset. Teams owning a product which will be retired soon, have little to no incentive to invest in further improving its engineering quality. Even more, as usage drops, they might start to remove approaches which are seen as obsolete, such as ceasing continuous deployment (CD) or not fixing continuous integration (CI) issues, if there are few changes happening in the codebase.

Team dynamics

- **An engineering manager who does not understand modern engineering practices.** The engineering manager leading the development team has a huge impact on its dynamics. If they have no experience with engineering practices like automated testing, [well-executed migrations](#), staged rollouts, short iteration cycles and so on, then the team is totally reliant on engineers educating this manager and convincing them to support these practices.

- **Product / sales running the show.** As I write in [Working with product managers as an engineering manager](#), a good relationship should be a partnership. However, many engineering managers have no such partnerships. In cases where product or sales is running the show and by extension the engineering team, the focus on engineering quality drops. This dynamic is common when the engineering team either has no manager, or has a hands-off or inexperienced engineering manager.
- **No senior / experienced engineers.** A team tends to ignore good engineering practices when no engineers on that team understand the importance of these practices, first-hand. Over time, they'll learn why those practices matter, but this learning might take years.
- **Team stretched too thinly.** If the team has too many things on its plate, there's a chance engineers will prioritize work related to engineering excellence last.
- **Outages don't have much business impact.** Engineering best practices tend to make products more resilient against mistakes. But what if the mistakes don't really matter; are they still worth investing in fixing? For teams with very few customers who don't mind if the product doesn't work as expected, sometimes, it might make no sense to invest much in best practices. Examples include building tools for a small number of support engineers, or teams which build custom reports for the finance team, whose requirements change all the time.
- **Ever-changing requirements.** On teams with no strong product manager, and where requirements change continuously, doing any automated testing may be a waste of time. Code that starts out as clean might become filled with "if/else" branches added as new use cases are introduced.
- **Deadlines set outside the team.** If the team has to meet unreasonable deadlines that were set without input from members, this is a bad situation. I do not recommend going along with situations like this, yet many teams still do. Engineering investment is the first thing that tends to be cut in pursuit of short-term wins.
- **Pressure to move on to the next thing.** On teams where it's not part of the culture to wrap-up projects, there's a risk the follow-up engineering work never gets done. Instead of spending the last part of a project on things like adding more

automated tests, improving build time, updating the documentation and so on, instead the engineers might be pushed to work on the next project, already.

- **The key engineer left.** In teams that are overly reliant on one or two engineers to ensure engineering quality, what happens if they depart? The bar often drops and the team reverts to working with no senior or experienced engineers onboard. In all fairness, there is often also a hands-off engineering manager involved, who's failing to advocate for sensible engineering practices.

Organizational dynamics

- **Acquisition.** The product your team works on might have come from the acquisition of a startup, which had different expectations for code quality than what is common at the larger company. It will take time for them to catch up.
- **Misaligned leadership incentives.** If the engineering manager and product manager are heavily incentivized to ship business impact quickly, and aren't penalized for outages or regressions, they might push to remove engineering quality gatekeeping. They may have incentives to only focus on short-term wins, ignoring mid and long-term impacts on reliability and maintainability.
- **Engineering quality doesn't help with promotions.** In organizations where impact is what gets engineers promoted – even if they take shortcuts with engineering approaches – it should be no surprise when good practices are followed less rigorously. In tech companies where engineering competencies list engineering excellence as a requirement for each level, the taking of shortcuts tends to be less of an issue. However, there are companies which don't define engineering excellence expectations – and also ones which do but then ignore them during promotions.
- **Different engineering quality bars in teams across the company.** All companies will have silos within the engineering organization. Some managers, directors and other engineering leaders will expect a much higher quality bar across engineering for their organizations, than others do. Back when I worked at Microsoft, there were some organizations with much higher reputations for engineering quality, than others. Back then, the MSN Messenger group had the internal perception of poor engineering quality and of being one to avoid working

in. The MSN group was later merged under Skype in 2013, and MSN Messenger discontinued in 2014.

You should not just accept at face value the above reasons, but you should understand the context first. Some of them are not excuses for a poor engineering culture. However, if you understand the context and the team's history, you'll be able to figure out how much a lack of engineering hygiene is contributing to lower standards.

Understanding *why* the team deprioritized engineering excellence will also help you figure out how you can best help to improve practices. You'll want to do this in a way that helps the team, is not treated as unwelcome, and to do so in a way that helps you and your career.

3. Handling the initial shock

You've joined The Company, the one you've looked up to so much, for so long. Having encountered a disappointing engineering culture, you're now feeling down. How can you best process this negative emotion?

Know that it's normal to feel disappointed about how poor things look from the inside. Many engineers who have joined well-known companies will tell you they felt similar. Talk with engineers at any large company and many will tell you how the code quality of their team is subpar, how they don't have time to invest in automation as much as they want to, or how many things are broken, which they did not predict before they joined. Here is a quote from [an article](#) written by former Amazon SDE [Ben Adam](#) on his shock when he started working there, about the lack of automation:

"The most surprising thing I encountered when joining was how manual most processes are. It blew my mind how many business critical processes were managed with excel spreadsheets being shared via email chains. It is incredible how flexible and effective Excel is for such a wide variety of use-cases."

Don't start fixing things on day one, start by shipping things on day one. Let's say you join a team which doesn't have any unit tests in its codebase, but you think this is a must-have. Should you start to commit these tests?

Hold off.

Instead of bringing your old habits and trying to force them on your new team, start by working the way the team already works and ship things into production the same way everyone else does. This way, you'll both get onboarded faster, but, more importantly you'll gain people's trust. As you onboard and gain trust, you will also learn more about why things are the way they are.

Are you used to one specific type of engineering excellence, and want to see the exact same implementation from your new team? Engineers with less diverse experience often go looking for the same type of setup for engineering excellence they already know. For example, if they previously worked at a startup which did unit testing and code reviews, they'll assume the new company must have a poor engineering culture if those things are missing.

While this assessment usually holds true, there are cases when teams still have a decently high bar, even if they are missing some best practices. It's typically engineers with experience across multiple domains who start to worry less about the specific practices and focus instead on whether things are actually hurting the team, or if the team actually just does things differently than they are used to.

Data engineer William Saar puts it like this:



William Saar
@saarw

[@GergelyOrosz](#) It's a balance. Easy for teams to focus too much on form rather than delivering value

Mid-experienced people (around 5 years) run the greatest risk of being overly concerned with form. Perhaps fear of stepping outside a small domain they've mastered keeps them polishing it

7:32 AM · Apr 8, 2022

32 Likes 2 Retweets

Learn about the history of your team. How did it start? What was the first version like? What were the challenges it faced? What are the biggest problems now?

It's hard to understand the present state of things if you don't know how it came about. I find that many poor engineering decisions are ones the team is well aware of, and that come from a past decision that didn't seem like a big deal at the time, but which has now turned into a bigger problem that's hard to fix.

For example, at Uber backend teams had the option to choose Go or Java as their main language. Several teams chose Go because it was a more modern language. Many realized only later how many libraries, frameworks and tools are either missing or are incomplete, compared to Java.

In the mid-term, opting for Go impacted engineering excellence and engineers coming from Java-heavy teams often found the approaches more bareboned. Understanding the history made it easier for them to emphasize with why this happened, and helped them accept that it would be a step-by-step process to get to a similar level of tooling as they were used to with Java services.

Ask your teammates for their thoughts on the engineering quality. Instead of assuming that no one cares about engineering excellence, ask them about it. They'll probably surprise you by seeing the same issues and being just as frustrated as you are.

Talk to engineers outside your immediate team. Before you decide that your team has a poor engineering culture, figure out what other nearby teams are doing and how it compares to yours. Do they have practices you are missing? Are they spending more effort on engineering excellence? Why? Why not?

Another benefit of talking with other teams is that you build a network. If you end up unable to push for meaningful change in your team, you'll find it easier to transfer to one that is more aligned with your own standard of engineering quality.

4. Improving the engineering culture

Pick up the rhythm of the team and gain people's trust. Before you jump into making changes that improve the quality of the codebase, first get onboarded and up to speed.

I once worked on a team which a senior engineer joined, and from the first week this person complained about our "poor practices." The team rolled their eyes, as everyone knew this already, but there were more important things to do, including getting ready to ship a major new feature.

The new engineer didn't do any work related to the new feature. They didn't complete even the simple tasks assigned to them. They were busy drawing up diagrams about how to change development to have a higher quality bar.

In the end, the team staged a mini revolt against the new engineer. A colleague put it in words to this effect:

"This person might have experience, but hasn't helped the team a single bit, struggles to complete their tasks, and is still lecturing us on what we should be doing?"

This new joiner was right in spotting issues that needed to be fixed, but they failed to build credibility within the team. Without credibility and support from others, it's an uphill battle to make change.

Don't be like this engineer who others see as someone who's all talk; prove you can do the work, just like any other team member. Once you've done so, your peers will be more open to your ideas.

Start with a small change. Instead of going all-in on improving practices, start with something less ambitious. For example, if the team has slow build times, instead of diving in to fix it all, just add logging to collect the statistics. Once that's in, make a change that might help; this could be enabling the caching of artefacts, or improving the build step that takes the longest time.

Similarly, when improving testing practices, start by adding a few tests, at first. Socialize the idea with the team and lead by example.

Explain the positive impact of the improvements. With a few improvements under your belt, share how these improvements are making a difference, so all the team knows. This is where having data is useful.

For build times, the time saved is obvious per build. However, instead of just sharing how you cut the build time from 30 to 15 minutes, do the math on how many people-hours you've saved per month or per year, with this change.

For improvements like testing or code reviews, the impact might not be visible immediately. Look to make the link to the impact of your work, where you can. With tests, this could be stories of how you caught a bug that would otherwise have passed through code review, thanks to the new test suite.

Add engineering excellence work to the backlog. With a few improvements up and running, you're probably at the limit of the engineering excellence work you can achieve on the side. If there's enough momentum, now might be a good time to make the case to add larger pieces of work to the backlog of the team, to be [prioritized during planning](#).

Consider facilitating a session where engineers share ideas they have for improving the team's engineering quality. Try to estimate both the effort of the work, and the impact of doing it. When estimating impact, think in terms that are relevant for the business. Ask if introducing this process or approach would result in:

- A more reliable product?
- Less time spent on maintenance?
- Shorter time to onboard new engineers?
- Fewer mistakes made by engineers, and so fewer bugs and higher reliability?
- Less likely that engineers would quit the team thanks to poor practices? That is, lower attrition.
- Less time waiting between changes; higher iteration speed on the engineering team, less context switching and greater engineering productivity?

- Fewer outages, thanks to common issues being caught earlier? This means fewer out-of-hours alerts and a more reliable product.

With a list of improvements and their impacts, you should be able to have constructive conversations on making some of this work a priority.

Be flexible in your approach. Don't assume that there is only The One Way to build a quality software product. I'd especially caution against pushing an approach you personally believe works best down others' throats. Instead, bring arguments team members can understand, and show the impact of the smaller changes you've already made to sell your new approaches and gain support across the team.

Build up a reputation of getting things done while improving quality. Don't forget your main role is to help the team execute on projects they signed up for. If you can do this reliably, while also leaving the code, the architecture and the engineering environment in a better state than you found it, you will build up a good reputation.

Who are the engineers much of the team looks to for guidance in improving engineering excellence? They're usually engineers with a track record of shipping things and with a pragmatic take on quality. They balance getting things done, leaving code in better shape than before, and spending time on engineering improvements and paying off tech debt.

5. Advice for avoiding surprises in your new job

Big Tech companies often consist of a huge number of teams, each with a different engineering culture and team dynamic. Choosing the right team to join – one that is not in a terrible state – is the best thing you can do for your career. As a software engineer at Meta summarizes:



Samhan
@Samhanknr

@GergelyOrosz Being good at team / company / problem selection is more important for a good career than people realize.

7:43 AM · Apr 8, 2022

66 Likes 2 Retweets



Samhan
@Samhanknr

@GergelyOrosz Do a lot of research before joining. If things are broken it can be an opportunity to change things and build up a reputation for fixing things. One should try to do that before moving on.

7:53 AM · Apr 8, 2022

Reverse interview your future manager and also a future teammate if you can, before joining. Ask them about engineering practices, team dynamics, the best things about the team and the biggest problems. Read more of [my advice on reverse interviewing](#).

There are a few companies at which reverse interviewing is not possible either because you don't know which team you will join – often the case at somewhere like Google – or because you choose the team after you onboard, which is the case at Meta with its Bootcamp. In these situations, I still suggest asking to talk with a tenured software engineer, to get a feel for the engineering culture from their point of view.

Takeaways

Keep your expectations in check when joining well-known companies. Big Tech is a large collection of small teams; all of which might work slightly differently and be at different stages of their product and team maturity.

Never stop learning about tools, processes and approaches which help to build and maintain quality software. However, be aware of the tradeoffs of each of these. In general, engineering excellence is a great long-term investment, and one which helps build more maintainable and reliable systems. When your team can afford to take this long-term view, it's a no-brainer to make the investment. However, many teams and situations call for shorter-term focus.

Be flexible in your approach to engineering excellence. There is no one way to do things well. When joining a new company, be curious and learn about the “whys” and “hows” of their approach. Put your judgement aside for the first few weeks and months, and you might learn counter-intuitive approaches that somehow still work.

Don’t forget you’re not hired to always write the highest quality code, or produce the most immaculate architecture. This is not to say that you should ever write bad code. However, sometimes a pragmatic, “good enough” approach goes further than striving for perfection.



Gergely Orosz
@GergelyOrosz

What many sw engineers don't realize:

The majority of jobs do not hire you to write the highest quality code or produce the cleanest architecture.

They hire you to solve their business problems very efficiently. Sometimes this means high-quality code. Sometimes not at all.

2:31 PM · Apr 10, 2022

8,544 Likes 1,247 Retweets

How did you like this week’s newsletter? 🤔

[Amazing](#) • [Great](#) • [Good](#) • [OK](#) • [So-so](#)



78 Likes

7 Comments



Write a comment...



Piotr Czech Apr 22, 2022 ❤️ Liked by Gergely Orosz

"For example, this is how many new organizations started within Uber, like UberEats, UberAir or UberRush. If they failed to get enough traction, they were shut down. In their early phases, all these teams wrote quick and dirty code and focused on shipping rapidly."

This approach is 50:50 because if you have a risky subject you should send the best team, every organization should have its own "special forces" to do dirty work and handle any subject. You don't need to sacrifice too much and you will get an answer that the product is feasible or not. For me, their "dirty code" is a mediocre solution rather than a top-notch long-term trade-off.

You can sacrifice only a small amount of your work rather than everything (mostly because they will use or create reusable parts for other systems). The best people cannot write useless code because it's very insufficient for them - they are "principle".

♡ LIKE (1) 💬 REPLY ...

3 replies by Gergely Orosz and others



Dimas Apr 20, 2022 ❤️ Liked by Gergely Orosz

Building a credibility of getting things done before even suggesting something is a solid advice 👍

♡ LIKE (1) 💬 REPLY ...

5 more comments...

